

ARIN5204: Reinforcement Learning

3.3 Advanced Policy Gradient

Dr. Long Chen

longchen@ust.hk

Assistant Professor, Dept. of CSE
The Hong Kong University of Science and Technology (HKUST)

May 5, 2026



- 1 Motivation
- 2 Natural Policy Gradients
- 3 TRPO, PPO

Some slides are from: Katerina Fragkiadaki (CMU), Sergey Levine (UCB), Emma Brunskill (Stanford), Shusen Wang (Meta).

① Motivation

② Natural Policy Gradients

③ TRPO, PPO

Recap: Actor-Critic

Actor-Critic Algorithm

1. Initialize policy parameters θ and critic parameters ϕ
2. Sample trajectories $\{\tau_i = \{s_t^i, a_t^i\}_{t=0}^T\}$ by deploying the current policy $\pi_{\theta}(a_t|s_t)$
3. Fit value function $V_{\pi}(s)$ by MC or TD estimation (update ϕ)
4. Compute action advantages $A_{\pi_{\theta}}(s_t^i, a_t^i) = r(s_t^i, a_t^i) + \gamma \cdot V_{\pi}(s_{t+1}^i) - V_{\pi}(s_t^i)$
5. $\nabla_{\theta} J(\theta) \approx \hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_t^i | s_t^i) \cdot A_{\pi_{\theta}}(s_t^i, a_t^i)$
6. $\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} J(\theta)$

(LOOP) GO TO STEP2

Q1: Choosing a Step Size for Policy Gradient

- Policy gradient algorithm is **stochastic gradient ascent**:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha_k \cdot \hat{g}_k$$

with step $d_k = \alpha_k \cdot \hat{g}_k$ (i.e., objective function gradient $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$)

- A common problem** may arise during updates:
 - If the step is **too large**, performance collapse is possible.

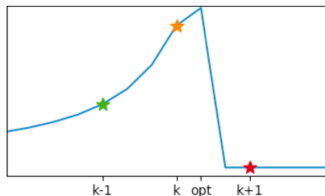


Figure: Policy parameters on x-axis and performance on y-axis. A bad step can lead to performance collapse, which may be hard to recover from.

Q1: Choosing a Step Size for Policy Gradient

- Policy gradient algorithm is **stochastic gradient ascent**:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha_k \cdot \hat{g}_k$$

with step $d_k = \alpha_k \cdot \hat{g}_k$ (i.e., objective function gradient $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$)

- **A common problem** may arise during updates:
 - If the step is **too large**, performance collapse is possible.
 - If the step is **too small**, progress is unacceptably slow
 - Thus, “**Right**” **step size** changes based on $\boldsymbol{\theta}$ is important

Q1: Choosing a “Right” Step Size for Policy Gradient

Q: The case is even worse for RL (Why?)

- Step too big:
 - Bad policy $\rightarrow$ data collected under bad policy $\rightarrow$ we cannot recover.
 - In SL, data does not depend on neural network weights
- Step too small:
 - Not efficient use of experience.
 - In SL, data can be trivially reused.

Q2: Parameter Space vs. Distribution Space

Policy gradient algorithms are stochastic gradient ascent:

- Consider a parameterized distribution π_{θ} and an objective $J(\theta)$ that depends on θ through π_{θ} and for which we want to take a gradient step.

$$\theta_{new} = \theta_{old} + d^*$$

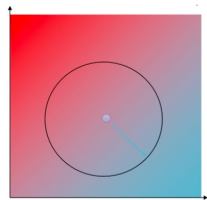
- Can be understood as:

$$\theta_{k+1} = \theta_k + \eta_k \cdot \mathbf{g}_k$$

where η_k is the learning rate, $\mathbf{g}_k = \nabla_{\theta} L(\theta_k)$

$$\theta_{k+1} = \arg \max_{\theta} \hat{L}_k(\theta), s.t., \|\theta - \theta_k\|_2^2 \leq \epsilon$$

$$\hat{L}_k(\theta) = L(\theta_k) + \mathbf{g}_k^T (\theta - \theta_k)$$



Q2: Parameter Space vs. Distribution Space

- **Gradient ascent**: the step in parameter space is determined by considering the Euclidean distance of the **parameter vectors** before and after the update:

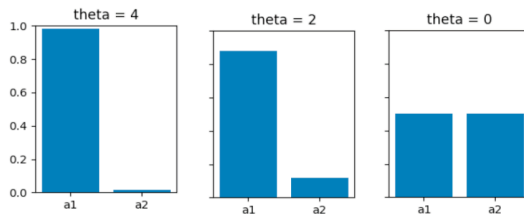
$$d^* = \arg \max_{\|d\| \leq \epsilon} J(\theta + d)$$

- Gradient descent in parameter space does not take into account the resulting distance in the (output) policy space between $\pi_{\theta_{old}}(s)$ and $\pi_{\theta_{new}}(s)$
- It is hard to predict how different is $\pi_{\theta_{new}}(s)$ from $\pi_{\theta_{old}}(s)$ (w.r.t. **Euclidean distance in parameter space**), i.e., **It is hard to pick the threshold ϵ .**

Example: Parameter Space vs. Distribution Space

- Consider a family of policies with parametrization:

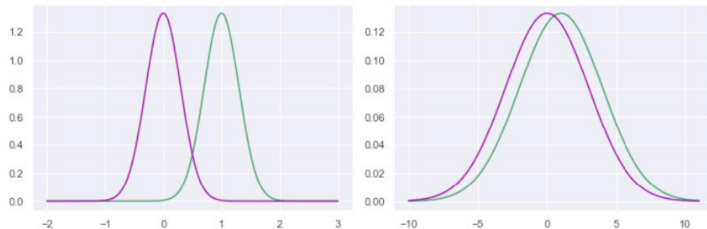
$$\pi_{\theta}(a) = \begin{cases} \sigma(\theta) & a = 1 \\ 1 - \sigma(\theta) & a = 2 \end{cases}$$



- The same parameter step $\Delta = -2$ changes the policy more or less dramatically depending on where in the parameter space we are.

Example: Parameter Space vs. Distribution Space

Example: Gaussian Distribution



- The left has $\mu_1 = 0, \mu_2 = 1$ and $\sigma_1 = \sigma_2 = 0.3$
- The right has $\mu_1 = 0, \mu_2 = 1$ and $\sigma_1 = \sigma_2 = 3.0$
- The Euclidean distances in the parameter space are 1.

- ① Motivation
- ② Natural Policy Gradients
- ③ TRPO, PPO

Gradient Descent in Distribution Space

- **Gradient ascent**: the step in parameter space is determined by considering the **Euclidean distance of the parameter vectors** before and after the update:

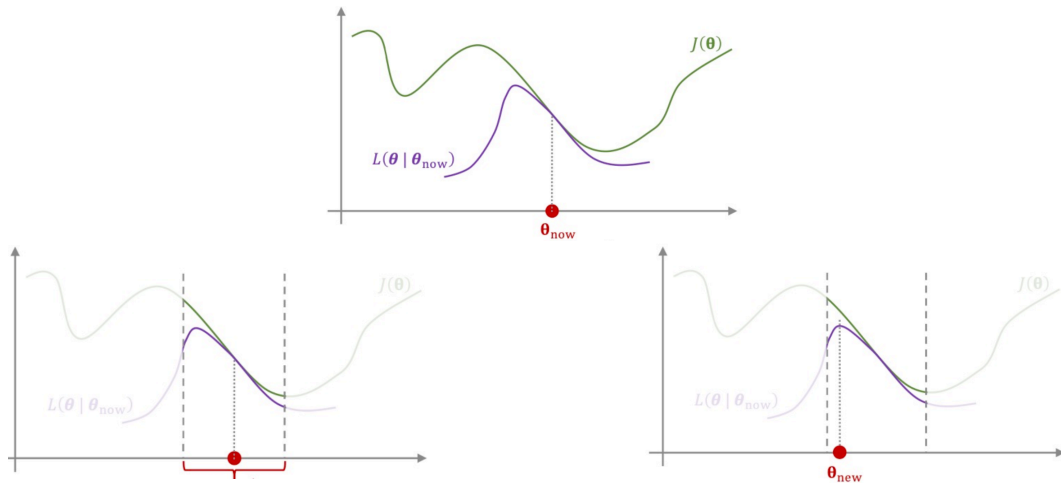
$$d^* = \arg \max_{\|d\| \leq \epsilon} J(\theta + d)$$

- **Euclidean distance in parameter space**
- It is hard to predict how different is $\pi_{\theta_{new}}$ from $\pi_{\theta_{old}}$. It is hard to pick the threshold ϵ
- **Natural gradient ascent**: the step in parameter space is determined by considering the KL divergence in the distribution before and after the update

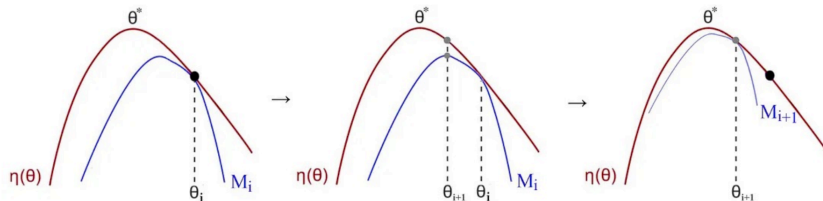
$$d^* = \arg \max_{D_{KL}(\pi_{\theta} \parallel \pi_{\theta+d}) \leq \epsilon} J(\theta + d)$$

- Easier to pick the distance threshold! (**Use trust region**)

Trust Region



Minorize-Maximization (MM) Algorithm



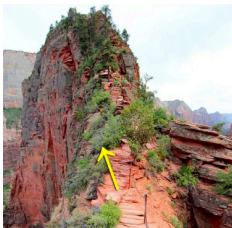
MM is an iterative method. For each iteration, we find a surrogate function M (the blue curve) that is:

- A lower bound function for η (real objective)
- Approximate η at the current policy, and
- Easy to optimize

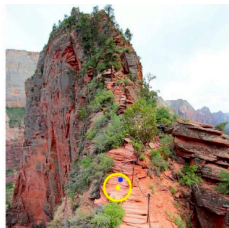
<https://jonathan-hui.medium.com/rl-the-math-behind-trpo-ppo-d12f6c745f33>

Intuition

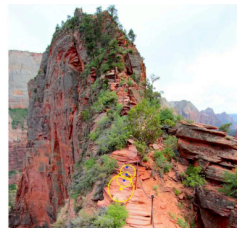
- **Gradient ascent**: if the learning rate is too high, the action may deviate from the real reward too much that, it turns into a disaster.
- In the **trust region**, we limit our search within a region. Such a region guarantees that its optimal policy will outperform the current one until it reaches a local and global optimum.



Line search
(like gradient ascent)



Trust region



Natural Policy Gradient

- **Training Objective:**

$$d^* = \arg \max_{D_{KL}(\pi_{\theta} || \pi_{\theta+d}) \leq \epsilon} J(\theta + d)$$

- **Unconstrained penalized objective** (using **Lagrange multiplier**):

$$d^* = \arg \max_d J(\theta + d) - \lambda (D_{KL}(\pi_{\theta} || \pi_{\theta+d}) - \epsilon)$$

Natural Policy Gradient

- Unconstrained penalized objective:

$$d^* = \arg\max_d J(\boldsymbol{\theta} + d) - \lambda(D_{KL}(\pi_{\boldsymbol{\theta}} || \pi_{\boldsymbol{\theta}+d}) - \epsilon)$$

- Taylor expansion:** first order for the objective and second order for the KL!

$$J(\boldsymbol{\theta} + d) \approx J(\boldsymbol{\theta}_{old}) + \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} \cdot d$$

$$\begin{aligned} D_{KL}(\pi_{\boldsymbol{\theta}} || \pi_{\boldsymbol{\theta}+d}) &\approx D_{KL}(\pi_{\boldsymbol{\theta}} || \pi_{\boldsymbol{\theta}}) \\ &\quad + d^T \nabla_{\boldsymbol{\theta}} D_{KL}(\pi_{\boldsymbol{\theta}} || \pi_{\boldsymbol{\theta}+d})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} \\ &\quad + \frac{1}{2} d^T \nabla_{\boldsymbol{\theta}}^2 D_{KL}(\pi_{\boldsymbol{\theta}} || \pi_{\boldsymbol{\theta}+d})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} \cdot d \end{aligned}$$

Taylor Expansion of KL (First-order)

Recall: $D_{KL}(p_{\theta_{old}} \| p_{\theta}) = \mathbb{E}_{x \sim p_{\theta_{old}}} \log\left(\frac{p_{\theta_{old}}(x)}{p_{\theta}(x)}\right)$

First order Taylor expansion of KL: $d^T \cdot \nabla_{\theta} D_{KL}(\pi_{\theta} \| \pi_{\theta+d})|_{\theta=\theta_{old}}$

$$\begin{aligned}\nabla_{\theta} D_{KL}(\pi_{\theta} \| \pi_{\theta+d})|_{\theta=\theta_{old}} &= -\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta_{old}}} \log p_{\theta}(x) |_{\theta=\theta_{old}} + \nabla_{\theta} \mathbb{E}_{x \sim p_{\theta_{old}}} \log p_{\theta_{old}}(x) |_{\theta=\theta_{old}} \\ &= -\mathbb{E}_{x \sim p_{\theta_{old}}} \nabla_{\theta} \log p_{\theta}(x) |_{\theta=\theta_{old}} \\ &= -\mathbb{E}_{x \sim p_{\theta_{old}}} \frac{1}{p_{\theta_{old}}(x)} \nabla_{\theta} p_{\theta}(x) |_{\theta=\theta_{old}} \\ &= \int_x p_{\theta_{old}}(x) \frac{1}{p_{\theta_{old}}(x)} \nabla_{\theta} p_{\theta}(x) |_{\theta=\theta_{old}} dx \\ &= \int_x \nabla_{\theta} p_{\theta}(x) |_{\theta=\theta_{old}} dx = \nabla_{\theta} \int_x p_{\theta}(x) |_{\theta=\theta_{old}} dx = 0\end{aligned}$$

Taylor Expansion of KL (Second-order)

Second order Taylor expansion of KL: $\frac{1}{2}d^T \nabla_{\theta}^2 D_{KL}(\pi_{\theta} || \pi_{\theta+d} |_{\theta=\theta_{old}}) \cdot d$

$$\begin{aligned}\nabla_{\theta}^2 D_{KL}(\cdot) &= -\nabla_{\theta}^2 \mathbb{E}_{x \sim p_{\theta_{old}}} \log p_{\theta}(x) |_{\theta=\theta_{old}} + \nabla_{\theta}^2 \mathbb{E}_{x \sim p_{\theta_{old}}} \log p_{\theta_{old}}(x) |_{\theta=\theta_{old}} \\&= -\mathbb{E}_{x \sim p_{\theta_{old}}} \nabla_{\theta}^2 \log p_{\theta}(x) |_{\theta=\theta_{old}} \\&= -\mathbb{E}_{x \sim p_{\theta_{old}}} \nabla_{\theta} \left(\frac{\nabla_{\theta} p_{\theta}(x)}{p_{\theta}(x)} \right) |_{\theta=\theta_{old}} \\&= -\mathbb{E}_{x \sim p_{\theta_{old}}} \left(\frac{\nabla_{\theta}^2 p_{\theta}(x) p_{\theta}(x) - \nabla_{\theta} p_{\theta}(x) \nabla_{\theta} p_{\theta}(x)^T}{p_{\theta}(x)^2} \right) |_{\theta=\theta_{old}} \\&= -\mathbb{E}_{x \sim p_{\theta_{old}}} \frac{\nabla_{\theta}^2 p_{\theta}(x) |_{\theta=\theta_{old}}}{p_{\theta_{old}}(x)} + \mathbb{E}_{x \sim p_{\theta_{old}}} \nabla_{\theta} \log p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)^T |_{\theta=\theta_{old}} \\&= \mathbb{E}_{x \sim p_{\theta_{old}}} \nabla_{\theta} \log p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)^T |_{\theta=\theta_{old}}\end{aligned}$$

Taylor Expansion of KL

$$\begin{aligned} D_{KL}(\pi_{\theta} || \pi_{\theta+d}) &\approx \\ D_{KL}(\pi_{\theta} || \pi_{\theta}) + d^T \nabla_{\theta} D_{KL}(\pi_{\theta} || \pi_{\theta+d})|_{\theta=\theta_{old}} &+ \frac{1}{2} d^T \nabla_{\theta}^2 D_{KL}(\pi_{\theta} || \pi_{\theta+d})|_{\theta=\theta_{old}} \cdot d \\ \nabla_{\theta}^2 D_{KL}(\pi_{\theta} || \pi_{\theta+d})|_{\theta=\theta_{old}} &= \mathbb{E}_{x \sim p_{\theta_{old}}} \nabla_{\theta} \log p_{\theta}(x) \nabla_{\theta} \log p_{\theta}(x)^T |_{\theta=\theta_{old}} \end{aligned}$$

- The **Fisher Information Matrix**

$$F(\theta_{old}) = \mathbb{E}_{x \sim p_{\theta_{old}}} [\nabla_{\theta} \log p_{\theta}(x) \cdot \nabla_{\theta} \log p_{\theta}(x)^T |_{\theta=\theta_{old}}]$$

- Can be approximated by sampling

$$F(\theta_{old}) \approx \frac{1}{N} \sum_{i=1, x^i \sim p_{\theta_{old}}}^N \left[\nabla_{\theta} \log p_{\theta}(x^i) \cdot \nabla_{\theta} \log p_{\theta}(x^i)^T |_{\theta=\theta_{old}} \right]$$

Solving the KL Constrained Problem

- **Unconstrained penalized objective:**

$$d^* = \arg\max_d J(\boldsymbol{\theta} + d) - \lambda(D_{KL}(\pi_{\boldsymbol{\theta}} \parallel \pi_{\boldsymbol{\theta}+d}) - \epsilon)$$

- First order Taylor expansion for the objective and second order for the KL!

$$\approx \arg\max_d J(\boldsymbol{\theta}_{old}) + \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} \cdot d - \frac{1}{2} \lambda (d^T \nabla_{\boldsymbol{\theta}}^2 D_{KL}(\pi_{\boldsymbol{\theta}} \parallel \pi_{\boldsymbol{\theta}+d})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} d) + \lambda \epsilon$$

- Substitute for the information matrix

$$\begin{aligned} &= \arg\max_d \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} \cdot d - \frac{1}{2} \lambda (d^T F(\boldsymbol{\theta}_{old}) d) \\ &= \arg\min_d -\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} \cdot d + \frac{1}{2} \lambda (d^T F(\boldsymbol{\theta}_{old}) d) \end{aligned}$$

Natural Gradient Descent

- Setting the gradient to zero:

$$0 = \frac{\partial}{\partial d} \left(-\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} \cdot d + \frac{1}{2} \lambda d^T F(\boldsymbol{\theta}_{old}) d \right)$$

$$= -\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} + \lambda F(\boldsymbol{\theta}_{old}) d$$

$$d = \frac{1}{\lambda} F^{-1}(\boldsymbol{\theta}_{old}) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}}$$

$$\frac{\partial X^T A X}{\partial X} = (A X + A^T X)^T$$

- The natural gradient:

$$\mathbf{g}_N = F^{-1}(\boldsymbol{\theta}_{old}) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} = F^{-1}(\boldsymbol{\theta}_{old}) \mathbf{g}$$

$$\boldsymbol{\theta}_{new} = \boldsymbol{\theta}_{old} + \alpha \cdot \mathbf{g}_N$$

- Q: How shall we choose stepsize along the natural gradient direction?

Stepsize Along the Natural Gradient Direction

- The natural gradient: $\mathbf{g}_N = F^{-1}(\boldsymbol{\theta}_{old}) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}} = F^{-1}(\boldsymbol{\theta}_{old}) \mathbf{g}$
 $\boldsymbol{\theta}_{new} = \boldsymbol{\theta}_{old} + \alpha \cdot \mathbf{g}_N$

- By the 2nd order Taylor expansion of KL:

$$D_{KL}(\pi_{\boldsymbol{\theta}_{old}} || \pi_{\boldsymbol{\theta}}) \approx \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}_{old})^T F(\boldsymbol{\theta}_{old})(\boldsymbol{\theta} - \boldsymbol{\theta}_{old}) = \frac{1}{2}(\alpha \mathbf{g}_N)^T F(\alpha \mathbf{g}_N)$$

- I want the KL between old and new policies to be at most ϵ
- Let's solve for the stepsize along the natural gradient direction

-

$$\frac{1}{2}(\alpha \mathbf{g}_N)^T F(\alpha \mathbf{g}_N) = \epsilon = \frac{1}{2}(\alpha F^{-1} \mathbf{g})^T F(\alpha F^{-1} \mathbf{g})$$

-

$$\Rightarrow \alpha = \sqrt{\frac{2\epsilon}{\mathbf{g}^T F^{-1} \mathbf{g}}}, \quad \Rightarrow \boldsymbol{\theta}_{new} = \boldsymbol{\theta}_{old} + \sqrt{\frac{2\epsilon}{\mathbf{g}^T F^{-1} \mathbf{g}}} \cdot F^{-1}(\boldsymbol{\theta}_{old}) \mathbf{g}|_{\boldsymbol{\theta}=\boldsymbol{\theta}_{old}}$$

Natural Policy Gradient

Algorithm 1 Natural Policy Gradient

Input: initial policy parameters θ_0

for $k = 0, 1, 2, \dots$ **do**

 Collect set of trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

 Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

 Form sample estimates for

- policy gradient $\hat{g}_k$ (using advantage estimates)
- and KL-divergence Hessian / Fisher Information Matrix $\hat{F}_k^{-1}$

 Compute Natural Policy Gradient update:

$$\theta_{k+1} = \theta_k + \sqrt{\frac{2\epsilon}{\hat{g}_k^T \hat{F}_k^{-1} \hat{g}_k}} \hat{F}_k^{-1} \hat{g}_k$$

end for

- ① Motivation
- ② Natural Policy Gradients
- ③ TRPO, PPO

Recap: Policy Gradient

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(a_{i,t} | s_{i,t}) \cdot \hat{A}_{i,t}^{\pi}$$

1. Estimate $\hat{A}_{\pi}(s_t, a_t)$ for current policy π
 2. Use $\hat{A}_{\pi}(s_t, a_t)$ to get improved policy π'
- (LOOP) GO TO STEP1

Q: Look familiar? Policy iteration algorithm!

1. Evaluate $A_{\pi}(s, a)$
 2. Set $\pi \leftarrow \pi'$
- (LOOP) GO TO STEP1

Policy Gradient as Policy Iteration

Q: How much exactly does the policy improve? $J(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}}(\tau)} [\sum_t \gamma^t r(s_t, a_t)]$

$$\begin{aligned} J(\boldsymbol{\theta}') - J(\boldsymbol{\theta}) &= J(\boldsymbol{\theta}') - \mathbb{E}_{s_0 \sim d} [V_{\pi_{\boldsymbol{\theta}}}(s_0)] = J(\boldsymbol{\theta}') - \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}'}(\tau)} [V_{\pi_{\boldsymbol{\theta}}}(s_0)] \\ &= J(\boldsymbol{\theta}') - \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t V_{\pi_{\boldsymbol{\theta}}}(s_t) - \sum_{t=1}^{\infty} \gamma^t V_{\pi_{\boldsymbol{\theta}}}(s_t) \right] \\ &= J(\boldsymbol{\theta}') + \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\gamma V_{\pi_{\boldsymbol{\theta}}}(s_{t+1}) - V_{\pi_{\boldsymbol{\theta}}}(s_t)) \right] \\ &= \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] + \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\gamma V_{\pi_{\boldsymbol{\theta}}}(s_{t+1}) - V_{\pi_{\boldsymbol{\theta}}}(s_t)) \right] \\ &= \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \gamma V_{\pi_{\boldsymbol{\theta}}}(s_{t+1}) - V_{\pi_{\boldsymbol{\theta}}}(s_t)) \right] \\ &= \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A_{\pi_{\boldsymbol{\theta}}}(s_t, a_t) \right] \end{aligned}$$

Policy Gradient as Policy Iteration

Understanding the Policy Improvement:

$$J(\boldsymbol{\theta}') - J(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A_{\pi_{\boldsymbol{\theta}}}(s_t, a_t) \right]$$

- Expectation under $\pi_{\boldsymbol{\theta}'}$
- Advantage under $\pi_{\boldsymbol{\theta}}$

$$\Delta J = \mathbb{E}_{\tau \sim p_{\boldsymbol{\theta}'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A_{\pi_{\boldsymbol{\theta}}}(s_t, a_t) \right] = \sum_t \mathbb{E}_{s_t \sim p_{\boldsymbol{\theta}'}(s_t)} \mathbb{E}_{a_t \sim \pi_{\boldsymbol{\theta}'}(a_t|s_t)} [\gamma^t A_{\pi_{\boldsymbol{\theta}}}(s_t, a_t)]$$

Policy Gradient as Policy Iteration

- Recall **importance sampling**:

$$\mathbb{E}_{x \sim p(x)}[f(x)] = \mathbb{E}_{x \sim q(x)} \left[\frac{p(x)}{q(x)} f(x) \right]$$

-

$$\begin{aligned} \Delta J &= \mathbb{E}_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A_{\pi_{\theta}}(s_t, a_t) \right] = \sum_t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} \mathbb{E}_{a_t \sim \pi_{\theta'}(a_t|s_t)} [\gamma^t A_{\pi_{\theta}}(s_t, a_t)] \\ &= \sum_t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} \mathbb{E}_{a_t \sim \pi_{\theta}(a_t|s_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \gamma^t A_{\pi_{\theta}}(s_t, a_t) \right] \end{aligned}$$

- Is it OK to use $p_{\theta}(s_t)$ instead?

Ignoring Distribution Mismatch?

$$\sum_t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} \mathbb{E}_{a_t \sim \pi_{\theta}(a_t|s_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \gamma^t A_{\pi_{\theta}}(s_t, a_t) \right]$$
$$\neq \sum_t \mathbb{E}_{s_t \sim p_{\theta}(s_t)} \mathbb{E}_{a_t \sim \pi_{\theta}(a_t|s_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \gamma^t A_{\pi_{\theta}}(s_t, a_t) \right] = \bar{A}(\theta')$$

- Q: Why do we want this to be true?
 - $J(\theta') - J(\theta) \approx \bar{A}(\theta') \Rightarrow \theta' \leftarrow \operatorname{argmax}_{\theta'} \bar{A}(\theta)$
 - Use $\bar{A}_{\pi}(s_t, a_t)$ to get improved policy π'
- Q: Is that true? And when?
 - Claim: $p_{\theta}(s_t)$ is close to $p_{\theta'}(s_t)$ when π_{θ} is close to $\pi_{\theta'}$

Bounding the Distribution Change

- **Claim:** $p_{\theta}(s_t)$ is close to $p_{\theta'}(s_t)$ when π_{θ} is close to $\pi_{\theta'}$
- Simple case: assume π_{θ} is a deterministic policy $a_t = \pi_{\theta}(s_t)$
- $\pi_{\theta'}$ is close to π_{θ} if $\pi_{\theta'}(a_t \neq \pi_{\theta}(s_t) | s_t) \leq \epsilon$
- $p_{\theta'}(s_t) = (1 - \epsilon)^t p_{\theta}(s_t) + (1 - (1 - \epsilon)^t) p_{mistake}(s_t)$
(**First term**) probability we made no mistakes
(**Second term**) some other distributions
- $|p_{\theta'}(s_t) - p_{\theta}(s_t)| = (1 - (1 - \epsilon)^t) |p_{mistake}(s_t) - p_{\theta}(s_t)| \leq 2(1 - (1 - \epsilon)^t)$
- Useful identity: $(1 - \epsilon)^t \geq 1 - \epsilon t$ for all $\epsilon \in [0, 1]$, i.e., $|p_{\theta'}(s_t) - p_{\theta}(s_t)| \leq 2\epsilon t$
- Not a great bound, but a bound!

Bounding the Objective Value

- $\pi_{\theta'}$ is close to π_{θ} if $|\pi_{\theta'}(a_t|s_t) - \pi_{\theta}(a_t|s_t)| \leq \epsilon$ for all s_t
- $|p_{\theta'}(s_t) - p_{\theta}(s_t)| \leq 2\epsilon t$
-

$$\begin{aligned}\mathbb{E}_{p_{\theta'}(s_t)}[f(s_t)] &= \sum_{s_t} p_{\theta'}(s_t) \cdot f(s_t) \geq \sum_{s_t} \pi_{\theta}(s_t) f(s_t) - |p_{\theta'}(s_t) - p_{\theta}(s_t)| \max_{s_t} f(s_t) \\ &\geq \mathbb{E}_{p_{\theta}(s_t)}[f(s_t)] - 2\epsilon t \max_{s_t} f(s_t)\end{aligned}$$

-

$$\begin{aligned}&\sum_t \mathbb{E}_{s_t \sim p_{\theta'}(s_t)} \mathbb{E}_{a_t \sim \pi_{\theta}(a_t|s_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \gamma^t A_{\pi_{\theta}}(s_t, a_t) \right] \\ &\geq \sum_t \mathbb{E}_{s_t \sim p_{\theta}(s_t)} \mathbb{E}_{a_t \sim \pi_{\theta}(a_t|s_t)} \left[\frac{\pi_{\theta'}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)} \gamma^t A_{\pi_{\theta}}(s_t, a_t) \right] - \sum_t 2\epsilon t C\end{aligned}$$

where $C : O(Tr_{max})$ or $O(\frac{r_{max}}{1-\gamma})$

Where are we at so far

$$\boldsymbol{\theta}' \leftarrow \arg \max_{\boldsymbol{\theta}'} \sum_t \mathbb{E}_{s_t \sim p_{\boldsymbol{\theta}}(s_t)} \mathbb{E}_{a_t \sim \pi_{\boldsymbol{\theta}}(a_t|s_t)} \left[\frac{\pi_{\boldsymbol{\theta}'}(a_t|s_t)}{\pi_{\boldsymbol{\theta}}(a_t|s_t)} \gamma^t A_{\pi_{\boldsymbol{\theta}}}(s_t, a_t) \right]$$

Such that

$$|\pi_{\boldsymbol{\theta}'}(a_t|s_t) - \pi_{\boldsymbol{\theta}}(a_t|s_t)| \leq \epsilon$$

for small enough ϵ , this is guaranteed to improve $J(\boldsymbol{\theta}') - J(\boldsymbol{\theta})$

How do we optimize the objective?

$$\boldsymbol{\theta}' \leftarrow \operatorname{argmax}_{\boldsymbol{\theta}'} \sum_t \mathbb{E}_{s_t \sim p_{\boldsymbol{\theta}}(s_t)} \mathbb{E}_{a_t \sim \pi_{\boldsymbol{\theta}}(a_t|s_t)} \left[\frac{\pi_{\boldsymbol{\theta}'}(a_t|s_t)}{\pi_{\boldsymbol{\theta}}(a_t|s_t)} \gamma^t A_{\pi_{\boldsymbol{\theta}}}(s_t, a_t) \right]$$

Such that

$$\underline{|\pi_{\boldsymbol{\theta}'}(a_t|s_t) - \pi_{\boldsymbol{\theta}}(a_t|s_t)| \leq \epsilon}$$

(OLD)

$$D_{KL}(\pi_{\boldsymbol{\theta}'}(a_t|s_t) || \pi_{\boldsymbol{\theta}}(a_t|s_t)) \leq \epsilon$$

(NEW)

for small enough ϵ , this is guaranteed to improve $J(\boldsymbol{\theta}') - J(\boldsymbol{\theta})$

Trust Region Policy Optimization

TRPO = NPG + Linesearch + monotonic improvement theorem

Algorithm 3 Trust Region Policy Optimization

Input: initial policy parameters θ_0

for $k = 0, 1, 2, \dots$ **do**

Collect set of trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Form sample estimates for

- policy gradient $\hat{g}_k$ (using advantage estimates)
- and KL-divergence Hessian-vector product function $f(v) = \hat{H}_k v$

Use CG with n_{cg} iterations to obtain $x_k \approx \hat{F}_k^{-1} \hat{g}_k$

Estimate proposed step $\Delta_k \approx \sqrt{\frac{2\epsilon}{x_k^T \hat{F}_k x_k}} x_k$

Perform backtracking line search with exponential decay to obtain final update

$$\theta_{k+1} = \theta_k + \alpha^j \Delta_k$$

end for

Trust Region Policy Optimization

- Due to the quadratic approximation, the KL constraint maybe violated!
What if we just do a linear search to find the best stepsize, making sure:
 - Improving objective $\bar{A}_{\pi_{old}}(\pi)$
 - The KL constraint is not violated

Algorithm 2 Line Search for TRPO

Compute proposed policy step $\Delta_k = \sqrt{\frac{2\epsilon}{\hat{g}_k^T \hat{F}_k^{-1} \hat{g}_k}} \hat{F}_k^{-1} \hat{g}_k$
for $j = 0, 1, 2, \dots, L$ **do**
 Compute proposed update $\theta = \theta_k + \alpha^j \Delta_k$
 if $\bar{A}_{\pi_{old}}(\pi) \geq 0$ and $\bar{D}_{KL}(\theta || \theta_k) \leq \delta$ **then**
 accept the update and set $\theta_{k+1} = \theta_k + \alpha^j \Delta_k$
 break
 end if
end for

Proximal Policy Optimization

- Can I achieve similar performance without second-order information (**no Fisher matrix!**)
- **Adaptive KL Penalty**
 - Policy update solves unconstrained optimization problem

$$\boldsymbol{\theta}_{k+1} = \arg\max_{\boldsymbol{\theta}} \bar{A}_{\boldsymbol{\theta}_k}(\boldsymbol{\theta}) - \beta_k \bar{D}_{KL}(\boldsymbol{\theta} || \boldsymbol{\theta}_k)$$

- Penalty coefficient β_k changes between iterations to approximately enforce the KL-divergence constraint

Proximal Policy Optimization

- Can I achieve similar performance without second-order information (no Fisher matrix!)
- **Adaptive KL Penalty**
- **Clipped Objective**
 - New objective function: let $r_t(\boldsymbol{\theta}) = \pi_{\boldsymbol{\theta}}(a_t|s_t)/\pi_{\boldsymbol{\theta}_k}(a_t|s_t)$. Then

$$L_{\boldsymbol{\theta}_k}^{CLIP}(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim \pi_k} \left[\sum_{t=0}^T [\min(r_t(\boldsymbol{\theta}) \hat{A}_t^{\pi_k}, \text{clip}(r_t(\boldsymbol{\theta}), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{\pi_k})] \right]$$

where ϵ is a hyperparameter (maybe $\epsilon = 0.2$)

- Policy update is $\boldsymbol{\theta}_{k+1} = \arg\max_{\boldsymbol{\theta}} L_{\boldsymbol{\theta}_k}^{CLIP}(\boldsymbol{\theta})$

PPO: Adaptive KL Penalty

- Using several epochs of minibatch SGD, optimize the KL-penalized objective

$$L^{KL PEN}(\boldsymbol{\theta}) = \hat{E}_t \left[\frac{\pi_{\boldsymbol{\theta}}(a_t | s_t)}{\pi_{\boldsymbol{\theta}_{old}}(a_t | s_t)} \hat{A}_t - \beta \cdot D_{KL}(\pi_{\boldsymbol{\theta}_{old}}(\cdot | s_t) || \pi_{\boldsymbol{\theta}}(\cdot | s_t)) \right]$$

- Compute $d = \hat{E}_t [D_{KL}(\pi_{\boldsymbol{\theta}_{old}}(\cdot | s_t) || \pi_{\boldsymbol{\theta}}(\cdot | s_t))]$
 - If $d < \frac{d_{targ}}{1.5}, \beta \leftarrow \beta/2$
 - If $d > d_{targ} \times 1.5, \beta \leftarrow \beta \times 2$

PPO: Clipped Objective

Input: initial policy parameters θ_0 , clipping threshold ϵ

for $k = 0, 1, 2, \dots$ **do**

Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Compute policy update

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}(\theta)$$

by taking K steps of minibatch SGD (via Adam), where

$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathbb{E}_{\tau \sim \pi_k} \left[\sum_{t=0}^T \left[\min(r_t(\theta) \hat{A}_t^{\pi_k}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{\pi_k}) \right] \right]$$

end for

- Clipping prevents policy from having incentive to go far away from θ_{k+1}
- Clipping seems to work at least as well as PPO with KL penalty, but is simpler to implement

Extra Reading Materials

- Trust Region Policy Optimization. In ICML, 2015.
- Proximal Policy Optimization Algorithms. In arXiv, 2017.
- Reinforcement Learning of Motor Skills with Policy Gradients. Neural Network, 2008.
- Natural Actor-Critic. In ECML, 2005.
- A Natural Policy Gradient. In NIPS, 2001.
- Constrained Policy Optimization. In ICML, 2017.